Exercise 4, Part 2: The Hook

Session: 4 — Skills, Hooks & Automation
Duration: 17 minutes, plus a 5-minute closing round
Project: The same BookStore API, on the `session4-playground` branch.

Goal

In Part 1 you made `/commit` and `/changelog`. Now you wire a hook that runs `/changelog` after every commit, run the chain, watch it stay silent, repair it, and run it again.

Start here

- Same folder, same branch as Part 1. Everything you create goes in `bookstore/.claude/`.
- Prompts that start with the experiment tag work as in Part 1. Copy them exactly, tag included.
- Never edit `CLAUDE.md`.
- The *Done when* lines are your checklist. Stuck? Type *just tell me*.

Tasks

3. Set up the hook and predict

This task produces `.claude/hooks/run-changelog.py`, an entry in `.claude/settings.json`, and three predictions.

1. **Let Claude create the script**. A matcher only sees the tool name, `Bash` (*Practical Hook Examples*); the script looks at the command. It is Python, not a shell script, because many of you are on Windows. Paste this whole block:

```
[Exercise 4 experiment — execute directly, no leading questions.] Create
.claude/hooks/run-changelog.py with exactly this content, nothing added or changed:

import json
import sys

data = json.load(sys.stdin)
command = data.get("tool_input", {}).get("command", "")

if command.startswith("git commit"):
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PostToolUse",
            "additionalContext": "A git commit was just made. Run the /changelog skill to
update CHANGELOG.md."
        }
    }))
```

2. **Add the hook to your settings**. Create the file `.claude/settings.json` yourself (it does not exist yet), with exactly this content:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "python .claude/hooks/run-changelog.py"
          }
        ]
      }
    ]
  }
}
```

   On mac or linux, write `python3` instead of `python`. Not sure? Run `python --version`; if that fails, use `python3`.

3. **Write three predictions**, on paper or in a text file, not in the chat; the verifier asks for them. One word first: a hook **fires** when Claude Code starts your script after a Bash tool call. The script then decides whether it prints anything. So a hook can fire and print nothing. One line each:

    1. Does the hook fire when *you* type `/commit`?
    2. Does it fire when Claude decides to run `git commit` by itself?
    3. Does it fire on `git commit --amend`?

One note on the script: only this nested JSON shape reaches Claude. Plain text from a `PostToolUse` hook goes to the transcript only (*Hook Handlers and Decisions*).

Done when: both files exist and your three predictions are written.

When the trainer calls task 4, start it, whatever your predictions look like.

4. Run the chain: it stays silent, then repair it

The *chain* (*Combining Skills and Hooks*): commit → hook → Claude runs your changelog skill. This task produces `CHANGELOG.md` and a repaired script.

1. **Remove an endpoint and commit**. A removal, on purpose: your changelog has to put something under `Removed`. Paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] Remove the DELETE /api/books/{id} endpoint from the BookStore API: its route, its handler, its store method and its test.
```

    When Claude is done, type `/commit`.

2. **Nothing happens. That is the plan**. `/commit` ends with the command line it ran and says your hook printed nothing. (If not, ask: *Which command line did you run for the last commit?*) The line starts with `git add`, because `/commit` stages and commits in one line: `git add … && git commit …`. Your script gets that whole line as one string. It does not start with `git commit`, so `startswith` is `False` and the script prints nothing.

    The hook fired: Claude Code ran your script. The script stayed silent. A hook reads the **text** of a command, never what the command did. (If the terminal showed a hook error instead, `python` was not found: fix the command in `settings.json`.)

3. **Repair the script**. It must also match a `git commit` after `&&` or `;`. Paste this whole block:

```
[Exercise 4 experiment — execute directly, no leading questions.] Replace the content of
.claude/hooks/run-changelog.py with exactly this, nothing added or changed:

import json
import re
import sys

data = json.load(sys.stdin)
command = data.get("tool_input", {}).get("command", "")

parts = re.split(r"&&|\|\||;", command)
if any(part.strip().startswith("git commit") for part in parts):
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PostToolUse",
            "additionalContext": (
                "A git commit was just made. Invoke the changelog skill now, "
                "in this same turn. Do not ask the user for permission first."
            )
        }
    }))
```

   Two changes: the script cuts the line into parts and checks each part, and the message says *invoke it now; do not ask*. The old one read like advice, and Claude may answer advice with a suggestion and wait.

4. **Run the chain again**. Add one more comment line by hand and type `/commit`. Now `CHANGELOG.md` appears, with the removal under `### Removed`. If nothing happens, look at the message text first, not at the matching.

5. **Check your predictions**. Mark each one confirmed or wrong, with one line of evidence from the chat, or one line of reasoning for the two you could not test.

6. **Run the verifier**. Type `/verify-exercise 4`. It asks for your three predictions first and settles them, then grades both skills, the hook and `CHANGELOG.md`. It checks the changelog against the rules in your task 2 prompt: the heading, the `## Unreleased` section, the group order, the verb and the hash on every entry. Read its report before the closing round.

Done when: `CHANGELOG.md` exists with the removal under `### Removed`, your three predictions are marked, and the verifier is running.

Bonus (only if time remains)

Send your script a fake event, the way Claude Code does. Save this as `.claude/fake-event.json`:

```
{"tool_input":{"command":"git add . && git commit -m x"}}
```

Run this in your own terminal, not in Claude (`python3` on mac or linux):

```
python .claude/hooks/run-changelog.py < .claude/fake-event.json
```

The repaired script prints JSON. Change the command in `fake-event.json` to `git status`: it prints nothing.

Closing round

The trainer asks the room. Have these answers ready:

- Which prediction was wrong, and what showed it?
- What can a hook see, and what can it not see?
- Which words in your `description` do you trust to start the skill at the right moment? Which would you make sharper now?
- One thing you would tell someone who skipped this session.

After the closing round

Clean up the branch. Two lines, one at a time (PowerShell 5.1 cannot join them with `&&`):

```
git switch -
git branch -D session4-playground
```

The commits are gone. Your files are not: `CHANGELOG.md`, `.claude/skills/`, `.claude/hooks/` and `.claude/settings.json` are gitignored.

Back at work. The plugin **skill-creator** on Anthropic's official marketplace writes a first version of a skill for you: `/plugin marketplace add anthropics/claude-plugins-official`, then `/plugin` to install it, then `/reload-plugins`. Try it on a job you repeat every week and compare its draft with your task 2 file.